

00w3c 100 案例全部实现一遍——作业

1 js 高级

写在前面，区别一下 java 的 static 和 js 中的 prototype

区别 1:

Java 中的 static 是在方法区中的 static 区域，方法中是没有 this 这个指定对象的

Js 中的 prototype 中是可以使用 this 这个指定对象的变量的，谁调用的原型方法 this 就是那个具体对象

1.1 原型与原型链

1.1.1 原型（prototype）

```
<script>
//每一个函数都有一个prototype属性，它默认指向一个Object空对象
console.log(Date.prototype,typeof Date.prototype);//带有若干方法的object
function fun(){}
//默认指向一个Object空对象(没有我们的属性)
//constructor: fun()
//__proto__:Object
console.log(fun.prototype,typeof fun.prototype)

//原型对象中有一个属性，指向函数对象
console.log(Date.prototype.constructor===Date);
console.log(fun.prototype.constructor===fun);

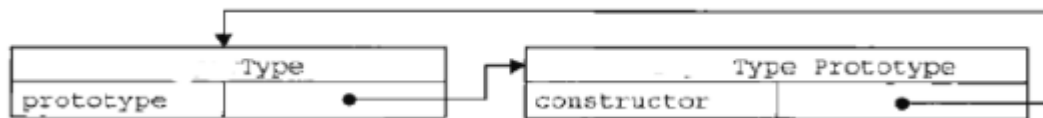
//给原型对象添加方法，实例对象可以访问
function Fun(){}

Fun.prototype.show=function(){
    console.log("fun--show()");
}
var temp=new Fun();
temp.show();
</script>
```

1.1.1.1 函数的 prototype 属性

A,每一个函数都有一个 prototype 属性，它默认指向一个 object 空对象（原型对象{}）

B,原型对象中有一个属性 constructor，他指向函数对象



```
//每一个函数都有一个prototype属性，它默认指向一个Object空对象
console.log(Date.prototype,typeof Date.prototype);//带有若干方法的object
function fun(){
//默认指向一个Object空对象(没有我们的属性)
//constructor: fun()
//__proto__:Object
console.log(fun.prototype,typeof fun.prototype)

//原型对象中有一个属性，指向函数对象
console.log(Date.prototype.constructor===Date);
console.log(fun.prototype.constructor===fun);
```

1.1.1.2 给原型对象添加属性（一般都是方法）

函数的所有实例对象自动拥有原型中的属性（方法）

```
//给原型对象添加方法，实例对象可以访问
function Fun(){

Fun.prototype.show=function(){
    console.log("fun--show()");
}
var temp=new Fun();
temp.show();
```

1.1.2 显示原型与隐式原型

A,每个函数 function 都有一个 prototype，即显示原型（属性）

b,每个实例对象都有一个__proto__，可称为隐式原型（属性）

c，对象的隐式原型的值为其对应构造函数的显示原型的值

d，总结

- 1， 函数的 prototype 属性： 在定义函数的时候自动添加，默认是一个空 object 对象
- 2， 对象的__proto__属性： 创建对象时自动添加的，默认值为构造函数的 prototype 属性值
- 3， 程序员能够直接操作显示原型，但是不能直接操作隐式原型（ES6 之前）

```

<script>
//隐式原型和显示原型
//定义构造函数
function Fn(){//内部语句: this.prototype={}
}
//1,每个函数function都有一个prototype, 即显示原型属性, 默认指向一个空object对象
console.log(Fn.prototype);
//2,每个实例对象都有一个__proto__, 可以称为隐式原型
//创建对象
var fn=new Fn();//内部语句: this.__proto__=Fn.prototype
console.log(fn.__proto__);
//3, 对象的隐式原型的值为其对应构造函数的显示原型的值
console.log(Fn.prototype===fn.__proto__);//true
//给原型添加方法
Fn.prototype.test=function(){
  console.log('test()');
}
//通过实例调用原型的方法
fn.test();
</script>

```

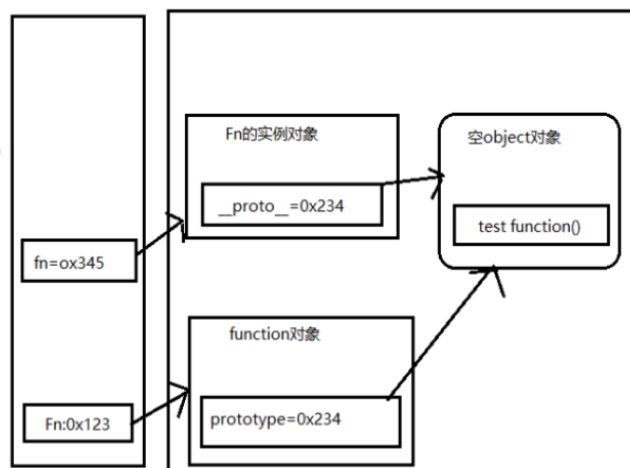
内存结构图

```

function Fn() {
}
var fn = new Fn() // fn是Fn的实例对象
console.log(Fn.prototype, fn.__proto__)
console.log(Fn.prototype===fn.__proto__)

//如何给所有实例对象添加方法? 通过原型
Fn.prototype.test = function () {
  console.log('test()')
}
fn.test()

```



1.1.3 原型链

1.1.3.1 原型链概念

原型链代码:

```

console.log(Object|)
console.log(Object.prototype)
function Fn() {
  this.test1 = function () {
    console.log('test1()')
  }
}
console.log(Fn.prototype)
Fn.prototype.test2 = function () {
  console.log('test2()')
}

var fn = new Fn()

fn.test1()
fn.test2()
console.log(fn.toString())

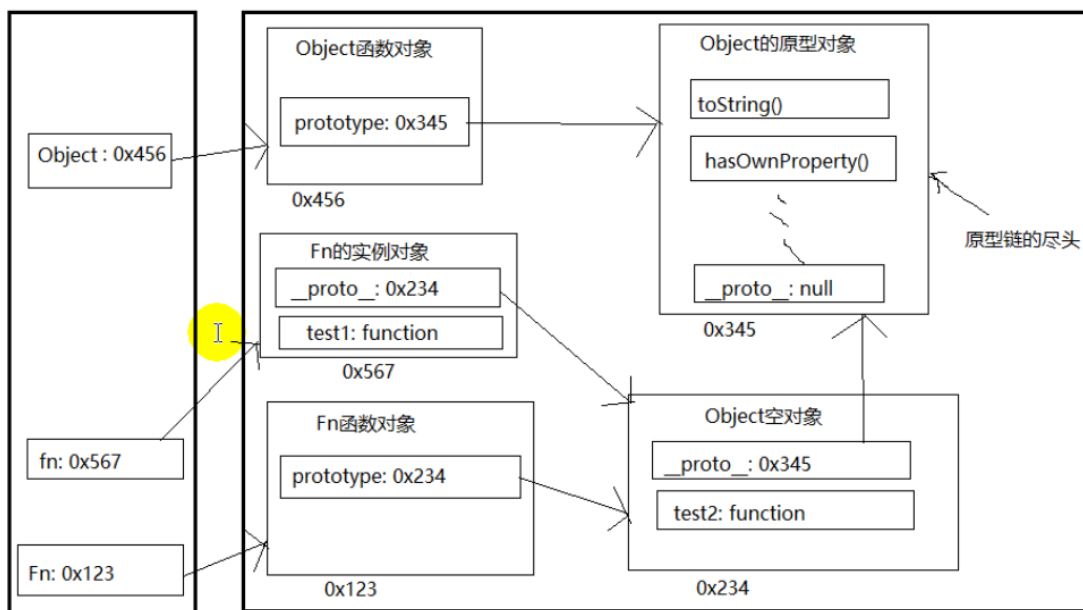
```

: 这里的 Object，是文档流自定义的系统内置函数，Object，函数就有显示原型

test3 ()

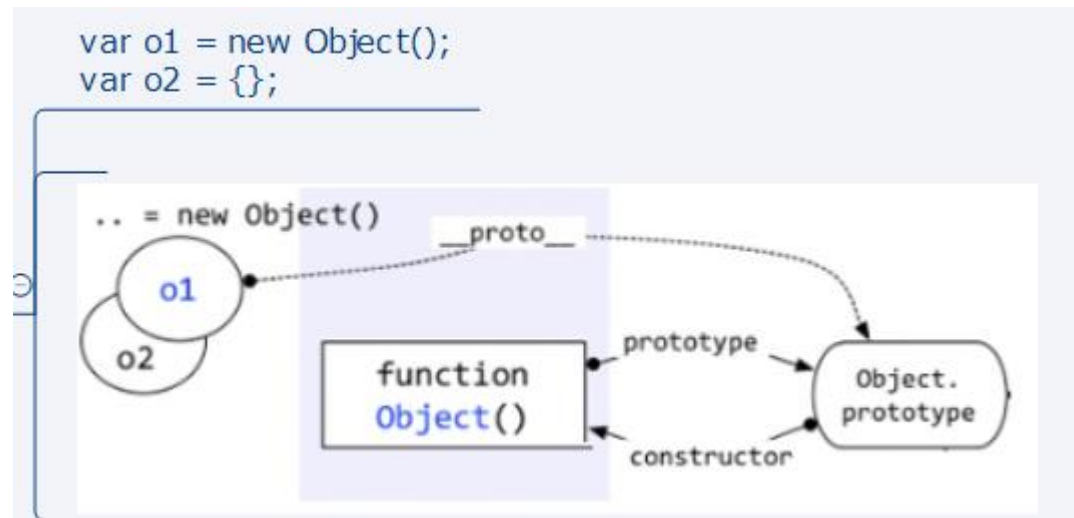
原型链图解:

提示: 这里的 Object: 0x456 指的是 function Object()的引用



讲述调用函数的时候的查找执行方法

案例 1：构造函数，原型，实例对象关系图



案例 2:

分析清楚

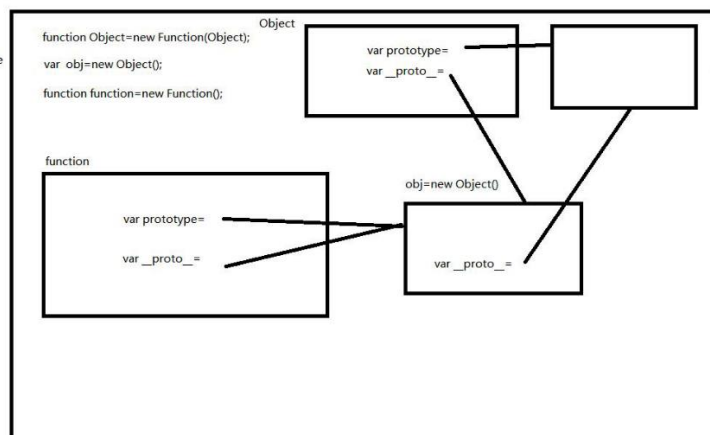
```
console.log(Object);
console.log(Object.__proto__ === Function.prototype); true
console.log(Function.__proto__ === Function.prototype); true
console.log(Function.prototype.__proto__ === Object.prototype); true
console.log(Object.prototype.__proto__);
```

::: 三个都是 true

```
console.log(Object.__proto__ === Function.prototype); true
console.log(Function.__proto__ === Function.prototype); true
console.log(Function.prototype.__proto__ === Object.prototype); true
```

对象的隐式原型的值为其对应构造函数的显示原型的值

- 1, Object被作为实例的时候, 他的构造函数是Function, 构造函数的显示原型== (构造函数new出来的实例) = (Function-Object) 的隐式原型——Object.__proto__ ==> true
- 2, Function作为实例的时候, Function是被Function本身new出来的, function Function=new Function();=>true
- 3, Function.prototype.__proto__也就是说, 这里把Function.prototype当做了实例, 他是一个Object的类型, Function.prototype是Object函数的实例, 函数的显示原型等于其实例隐式原型的值==> true

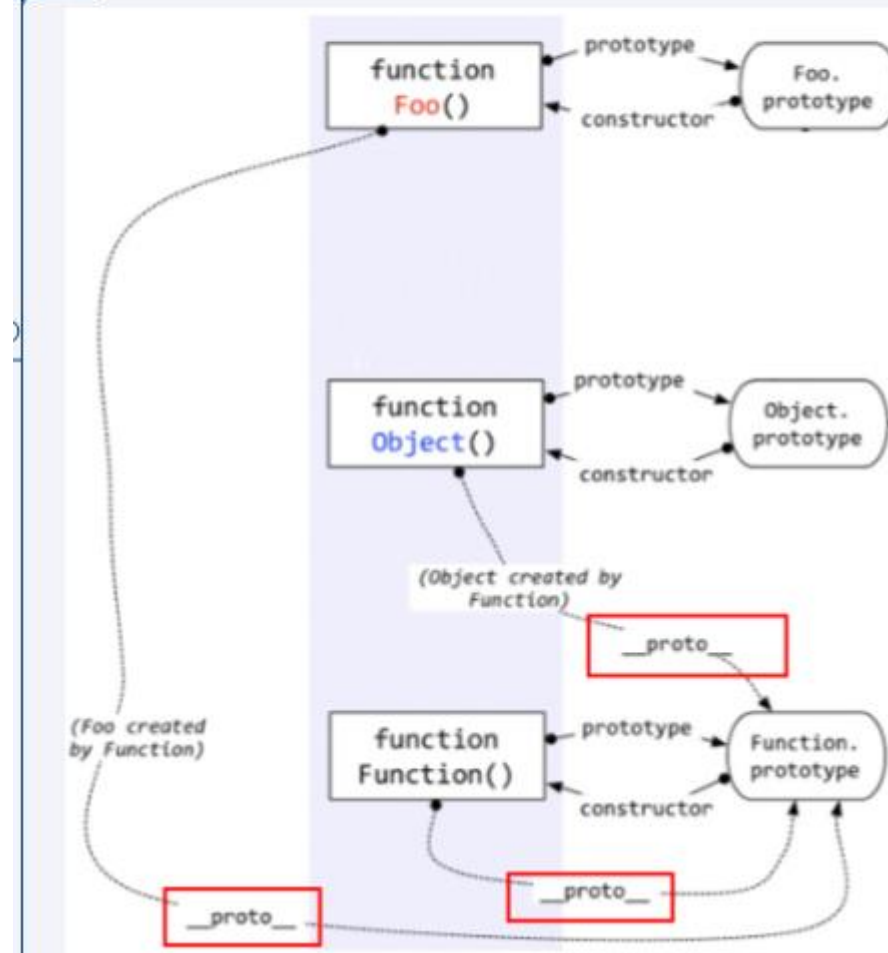


```
function Foo(){ }
```

```
var Foo=new Function();
```

```
Function=new Function()
```

所有函数的__proto__都是一样的



1.1.3.2 原型链细节补充

如果 B 函数的显示原型对象在 A 对象的原型链上，返回 `true`，否则返回 `false`

```

/*
1. 函数的显示原型指向的对象默认是空Object实例对象(但Object不满足)
*/
console.log(Fn.prototype instanceof Object) // true
console.log(Object.prototype instanceof Object) // false
console.log(Function.prototype instanceof Object) // true
/*
2. 所有函数都是Function的实例(包含Function)
*/
console.log(Function.__proto__ === Function.prototype)
/*
Object的原型对象是原型链尽头
*/
console.log(Object.prototype.__proto__) // null

```

1.1.3.3 原型链属性问题

```

//01 原型链，属性问题
/*
1, 读取对象的属性时，会自动到原型链中去查找
2, 设置对象的属性时，不会到原型链，如果当前对象中没有此属性，直接添加此属性并设置其值
3, 方法一般定义在原型中，属性一般通过构造函数定义在对象本身上
*/
function Fn(){}
Fn.prototype.a='xxxx';
var fn1=new Fn();
console.log(fn1.a,fn1);

var fn2=new Fn();
fn2.a='zzz';
console.log(fn1.a,fn2.a,fn2);

```

```

xxxx ▼ Fn ⓘ
  ▼ __proto__: Object
    a: "xxxx"
    ► constructor: Fn()
    ► __proto__: Object
-----
xxxx zzz ▼ Fn ⓘ
  a: "zzz"
  ▼ __proto__: Object
    a: "xxxx"
    ► constructor: Fn()
    ► __proto__: Object

```

一般案例

```
function Person(name, age) {  
  this.name = name  
  this.age = age  
}  
Person.prototype.setName = function (name) {  
  this.name = name  
}  
var p1 = new Person('Tom', 12)  
p1.setName('Bob')  
console.log(p1)  
  
var p2 = new Person('Jack', 12)  
p2.setName('Cat')  
console.log(p2)
```

04

```
▼ Person {name: "Bob", age: 12} ⓘ  
  age: 12  
  name: "Bob"  
  __proto__: Object  
    ▶ setName: function (name)  
    ▶ constructor: function Person(name, age)  
    ▶ __proto__: Object
```

04

```
▼ Person {name: "Cat", age: 12} ⓘ  
  age: 12  
  name: "Cat"  
  __proto__: Object  
    ▶ setName: function (name)  
    ▶ constructor: function Person(name, age)  
    ▶ __proto__: Object
```

1.1.4 instanceof

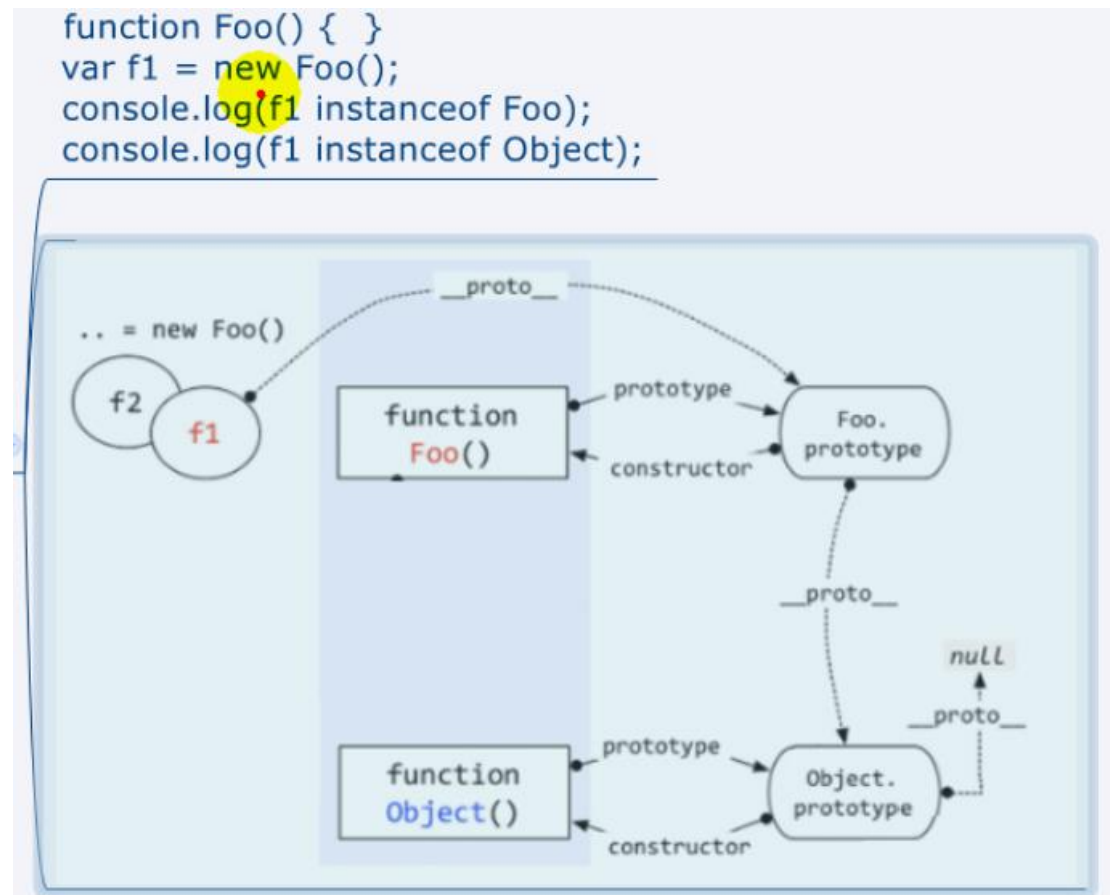
1, Instanceof 是如何判断的?

表达式: A instanceof B

如果 B 函数的显示原型对象在 A 对象的原型链上，返回 true，否则返回 false

2, Function 是通感 new 自己产生的实例

案例一: true、true



案例二:

```
/*
案例2
*/
console.log(Object instanceof Function) // true
console.log(Object instanceof Object) // true
console.log(Function instanceof Function) // true
console.log(Function instanceof Object) // true

function Foo() {}
console.log(Object instanceof Foo) // false
```

解释第二条执行顺序:

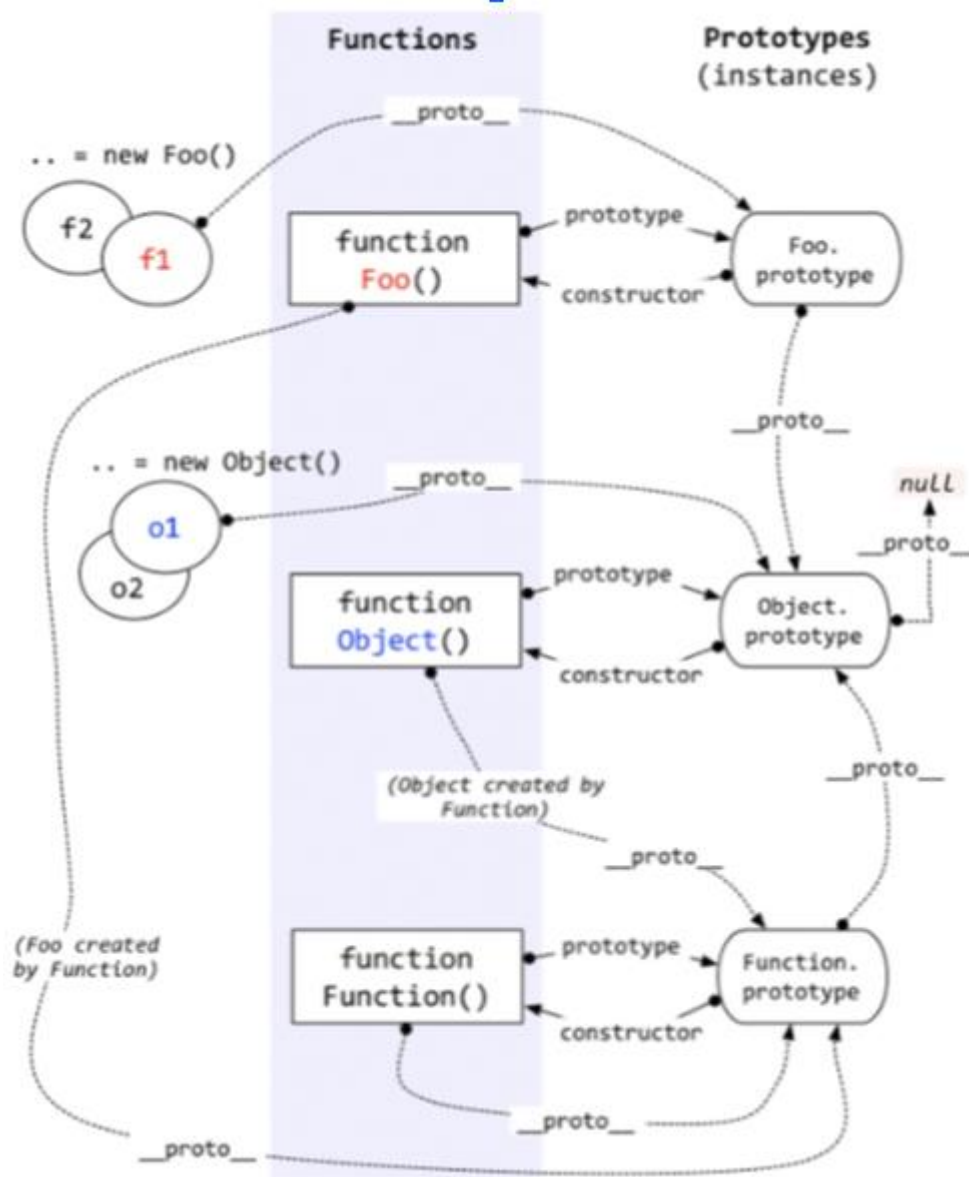
True 的条件: B 函数的显示原型对象在 A 对象的原型链上

Object instanceof object a of b a 是实例 b 是类型

B 是构造函数, 构造函数看显示原型属性 prototype, object 类型的 prototype 为 Object.prototype

A 是 object 构造函数的实例对象, 所以实例对象看隐式原型属性, 即是 __proto__ 属性, object

是 Function Object () 的实例，所以 object 的隐式原型属性是 Function 构造函数的显示原型属性，Function 的显示原型属性是空对象，这个空对象本身是 Object (类型) 函数的实例，既然是实例那么这个空对象的隐式原型对象就是 Function Object () 的显示原型对象，即 b 的原型属性，所以返回 true



面试题:

```
/*
测试题1
*/
```

```
function A () {
}
A.prototype.n = 1

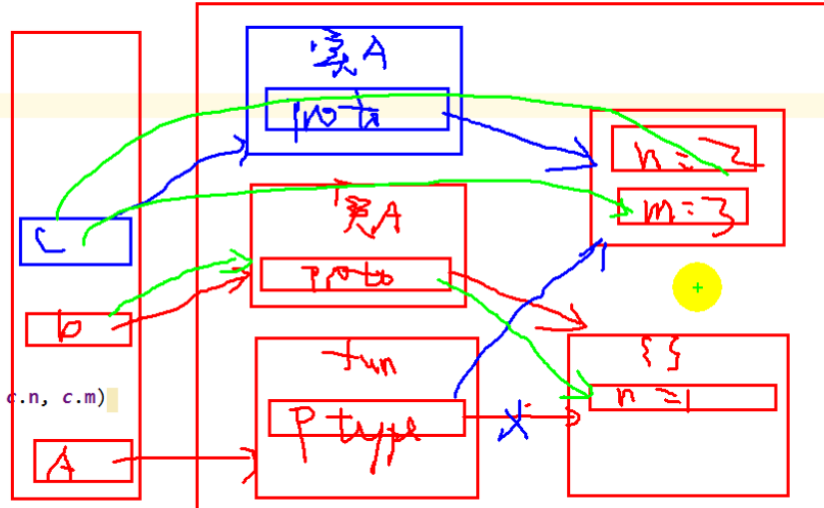
var b = new A()

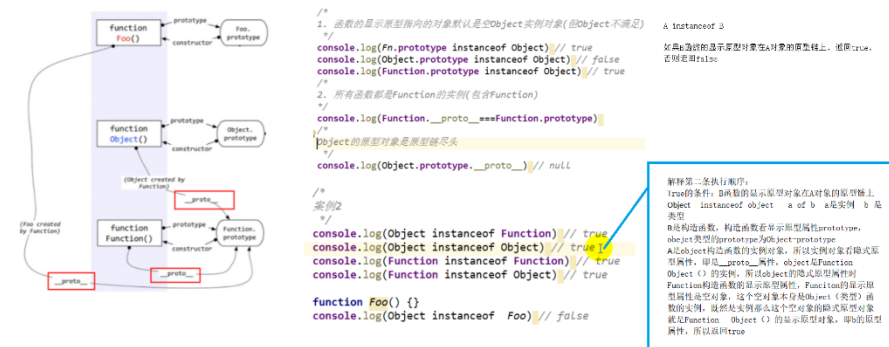
A.prototype = {
  n: 2,
  m: 3
}

var c = new A()
console.log(b.n, b.m, c.n, c.m)
```

```
/*
测试题2
*/
```

```
var F = function(){}
Object.prototype.a = function(){
  console.log('a()')
}
Function.prototype.b = function(){
  console.log('b()')
}
var f = new F()
f.a()
f.b()
F.a()
F.b()
```





1.2 执行上下文与执行上下文栈

1.2.1 变量提升与函数提升---? 原理?

1. 变量声明提升

- * 通过var定义(声明)的变量, 在定义语句之前就可以访问到
- * 值: undefined

2. 函数声明提升

- * 通过function声明的函数, 在之前就可以直接调用
- * 值: 函数定义(对象)

3. 问题: 变量提升和函数提升是如何产生的?

```
var a = 3
function fn () {
  console.log(a)
  var a = 4
}
fn()
```

```
console.log(b) //undefined 变量提升
fn2() //可调用 函数提升
// fn3() //不能 变量提升
```

```
var b = 3
function fn2() {
  console.log('fn2()')
}
```

```
var fn3 = function () {
  console.log('fn3()')
}
```

1.2.2 执行上下文

1. 代码分类(位置)
 - * 全局代码
 - * 函数(局部)代码
2. 全局执行上下文
 - * 在执行全局代码前将window确定为全局执行上下文
 - * 对全局数据进行预处理
 - * var定义的全局变量==>undefined, 添加为window的属性
 - * function声明的全局函数==>赋值(fun), 添加为window的方法
 - * this==>赋值(window)
 - * 开始执行全局代码
3. 函数执行上下文
 - * 在调用函数, 准备执行函数体之前, 创建对应的函数执行上下文对象(虚拟的, 存在于栈中)
 - * 对局部数据进行预处理
 - * 形参变量==>赋值(实参)==>添加为执行上下文的属性
 - * arguments==>赋值(实参列表), 添加为执行上下文的属性
 - * var定义的局部变量==>undefined, 添加为执行上下文的属性
 - * function声明的函数 ==>赋值(fun), 添加为执行上下文的方法
 - * this==>赋值(调用函数的对象)
 - * 开始执行函数体代码

全局执行上下文

```
console.log(a1, window.a1)
a2()
console.log(this)

var a1 = 3
function a2() {
  console.log('a2()')
}
console.log(a1)
```

函数执行上下文

```

// 函数执行上下文
function fn (a1) {
  console.log(a1) // 2
  console.log(a2) // undefined
  a3() // a3()
  console.log(this) // window
  console.log(arguments) // 伪数组(2,3)
}

var a2 = 3
function a3() {
  console.log('a3()')
}

fn(2, 3)

```

1.2.3 执行上下文栈

1. 在全局代码执行前, JS引擎就会创建一个栈来存储管理所有的执行上下文对象
2. 在全局执行上下文(window)确定后, 将其添加到栈中(压栈)
3. 在函数执行上下文创建后, 将其添加到栈中(压栈)
4. 在当前函数执行完后, 将栈顶的对象移除(出栈)
5. 当所有的代码执行完后, 栈中只剩下window

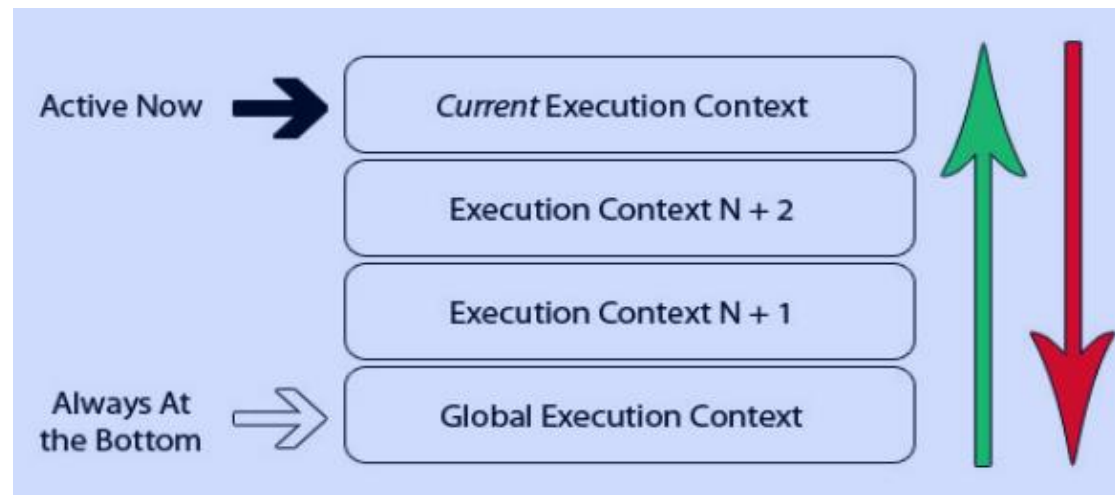


```

<script type="text/javascript">
    //1. 进入全局执行上下文
    var a = 10
    var bar = function (x) {
        var b = 5
        foo(x + b)
    }
    //3. 进入foo执行上下文
    var foo = function (y) {
        var c = 5
        console.log(a + c + y)
    }

    bar(10)
    //2. 进入bar函数执行上下文
</script>

```



1.2.4 案例讲解

案例一：

/*

整个过程产生了5个执行上下文

'globle begin:'+undefined

'foo() begin:' 1

'foo() begin:' 2

'foo() begin:' 3

foo() end: 3

foo() end: 2

foo() end: 1

'globle end:'+ 1

*/

console.log('globle begin:'+i);

var i=1;

foo(1);

function foo(i){

if(i==4){

return ;

}

console.log('foo() begin:'+i);

foo(i+1);

console.log('foo() end:'+i);

}

console.log('globle end:'+i);

案例二

```

//同时出现函数提升和变量提升时，先执行变量提升，再执行函数提升
function f(){}
var f;
console.log(typeof f);//function
//
if(!(b in window)){
    var b=1;
}
console.log(b);//undefined
/*
    var c;
    function c(c){
        console.log(c);
        var c=1;
    }
    c=1;
    c(2);//error!
*/
var c=1;
function c(c){
    console.log(c);
    var c=1;
}
c(2);//?

```

1.3 作用域与作用域链

1.3.1 作用域与执行上下文

理解作用域

1. 理解

- * 就是一块"地盘", 一个代码段所在的区域
- * 它是静态的(相对于上下文对象), 在编写代码时就确定了

2. 分类

- * 全局作用域
- * 函数作用域
- * 没有块作用域(ES6有了)

3. 作用

- * 隔离变量, 不同作用域下同名变量不会有冲突

//js没有块作用域

```
if(true){  
  var c=3;  
}
```

```
console.log(c);//3
```

作用域图例



作用域与执行上下文

1. 区别1

- * 全局作用域之外，每个函数都会创建自己的作用域，作用域在函数定义时就已经确定了。而不是在函数调用时
- * 全局执行上下文环境是在全局作用域确定之后，js代码马上执行之前创建
- * 函数执行上下文是在调用函数时，函数体代码执行之前创建

2. 区别2

- * 作用域是静态的，只要函数定义好了就一直存在，且不会再变化
- * 执行上下文是动态的，调用函数时创建，函数调用结束时就会自动释放

3. 联系

- * 上下文环境(对象)是从属于所在的作用域
- * 全局上下文环境==>全局作用域
- * 函数上下文环境==>对应的函数使用域

I

```
var a = 10,  
    b = 20;
```

全局作用域

全局上下文环境	
a	10
d	20
其他	

```
function fn(x) {  
    var a = 100,  
        c = 300;
```

fn作用域

fn(10) 上下文环境	
x	10
a	100
c	300
其他	

```
function bar(x) {  
    var a = 1000,  
        d = 4000;  
}
```

bar作用域

bar(100) 上下文环境	
x	100
a	1000
d	4000
其他	

```
bar(100);  
bar(200);
```

```
fn(10);
```

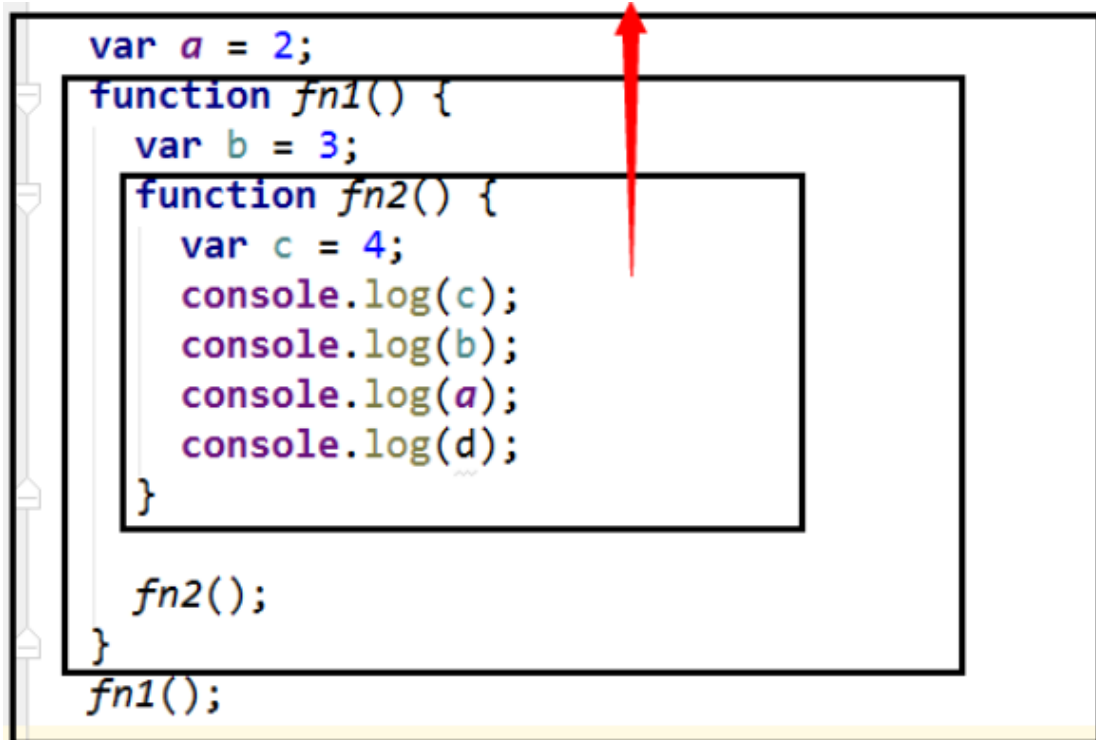
1.3.2 作用域链

1. 理解

- * 多个上下级关系的作用域形成的链，它的方向是从下向上的(从内到外)
- * 查找变量时就是沿着作用域链来查找的

2. 查找一个变量的查找规则

- * 在当前作用域下的执行上下文中查找对应的属性，如果有直接返回，否则进入2
- * 在上一级作用域的执行上下文中查找对应的属性，如果有直接返回，否则进入3
- * 再次执行2的相同操作，直到全局作用域，如果还找不到就抛出找不到的异常



1.3.3 案例

案例一

```
//函数在声明时作用域就已经被定好了，  
//所以如果是函数作为执行参数传入其他函数，  
//在执行这个被当成参数的函数时  
//他的作用域其实是声明时自己处于的作用域  
var x=10;  
function fn(){  
    console.log(x);  
}  
function show(fn){  
    var x=20;  
    fn();  
}  
show(fn);
```

案例二

对象中函数访问其他函数需要获取对象内存，再从这个实例对象内存中找到相关函数作用域，才能调用，本方法调用必须使用 this

```
var fn = function () {  
    console.log(fn)  
}  
fn()  
  
var obj = {  
    fn2: function () {  
        console.log(fn2)  
        //console.log(this.fn2)  
    }  
}  
obj.fn2()
```

1.4 闭包

1.4.1 引入

需求：点击时弹出对应的数组角标



```
<html>
  <head>
    <title>

    </title>

    <script>
      window.onload=function(){

        var btns=document.getElementsByClassName('btn');
        //方法一，这个出现的问题是每次点击都会打印 4，因为每次点击时，在
        执行事件响应函数的时候 i 已经被 for 遍历完了一遍
        /*for(var i=0,length=btns.length;i<length;i++){
          btns[i].onclick=function(){
            alert(i);
          }
        }*/
        //方法二，给点击的标签对象加脚标
        /*for(var i=0,length=btns.length;i<length;i++){
          btns[i].index=i+1;
          btns[i].onclick=function(){
            alert(this.index);
          }
        }*/
        //方法三，闭包
        for(var i=0,length=btns.length;i<length;i++){
          (function(i){
            var btn=btns[i];
            btn.onclick=function(){
              alert(i+1);
            }
          })(i);
        }

      }
    </script>
  </head>
  <body>
    <button class="btn">测试 1</button>
    <button class="btn">测试 2</button>
    <button class="btn">测试 3</button>
    <button class="btn">测试 4</button>
```

```
</body>
</html>
```

1.4.2 闭包的定义

1. 如何产生闭包?

* 当一个嵌套的内部(子)函数引用了嵌套的外部(父)函数的变量(函数)时, 并且外部函数被调用的时候就产生了闭包

2. 闭包到底是什么?

* 使用chrome调试查看

* 理解一: 闭包是嵌套的内部函数(绝大部分人)

* 理解二: 包含被引用变量(函数)的对象(极少数人)

* 注意: 闭包存在于嵌套的内部函数中

3. 产生闭包的条件?

* 函数嵌套

* 内部函数引用了外部函数的数据(变量/函数)

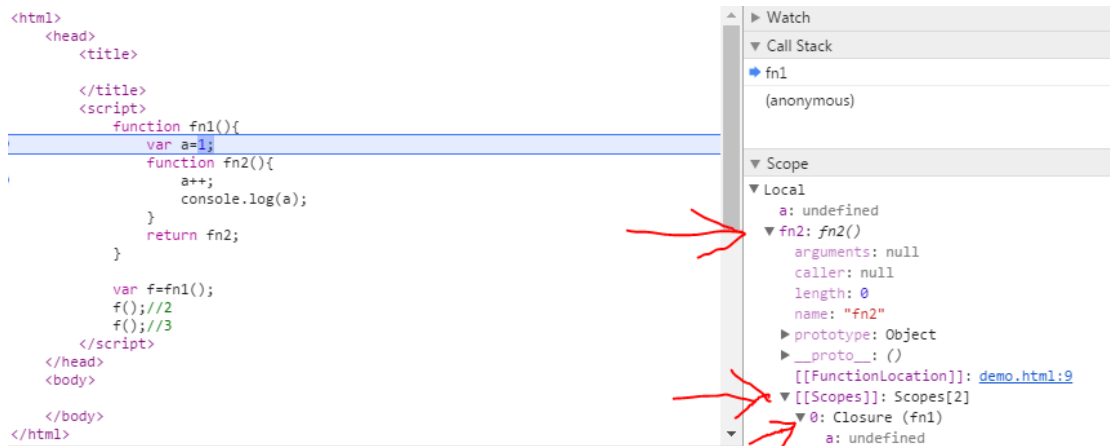
```
<script type="text/javascript">
  function fn1 () {
    var a = 2
    function fn2 () {
      console.log(a)
    }
  }
  fn1()
</script>
```

1.4.3 常见闭包

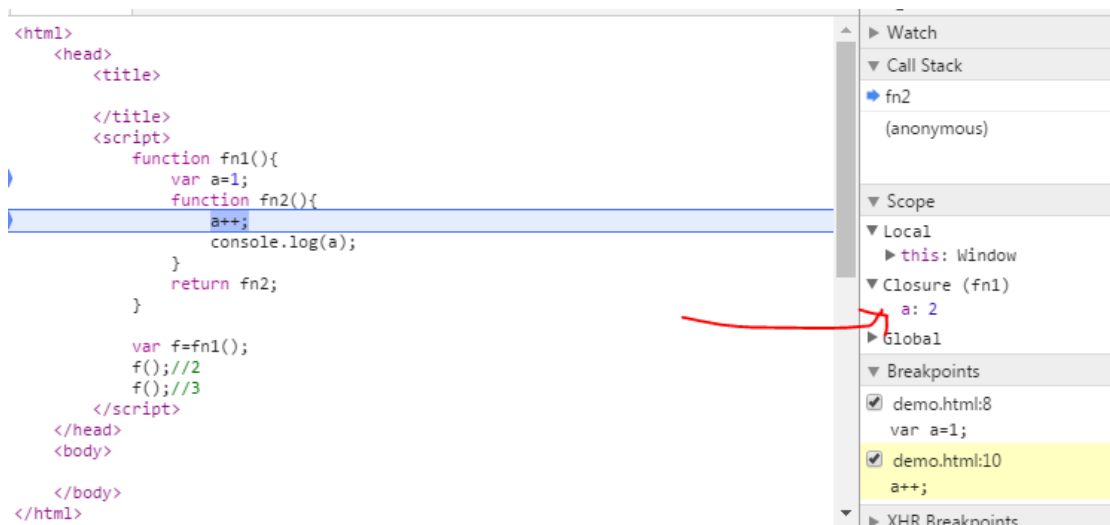
情况一: 将一个函数作为外部函数的返回值返回

```
function fn1() {
  var a = 2
  function fn2() {
    a++
    console.log(a)
  }
  return fn2
}
var f = fn1()
f() // 3
f() // 4
```

讲解:



第一次执行 fn() 时，在执行到 a=1; 的时候，因为函数提升机制，fn2（）内部又调用了 fn1（）的参数，所以 fn2（）内部产生一个闭包，closure（fn1）



第二次执行的时候，在同一个闭包中执行 fn2（）的方法，a 的值进行叠加，因此，只是函数执行，该函数不会释放资源，只会调用同一个闭包进行函数方法的执行

情况二：将函数作为实参传递给另外一个函数调用

```
function showDelay(msg,time){
  setTimeout(function(){
    alert(msg);
  },time);
}
showDelay('aaa',2000);
```

解释：其中的内部函数 function() 使用到了外部函数变量 msg，那么就在内部函数中产生一个包含 msg 的对象（闭包），如果将这个 function 去掉，那么在 showDelay 内部就不会产生一个闭包

1.4.4 闭包的作用

1. 使用函数内部的变量在函数执行完后，仍然存活在内存中(延长了局部变量的生命周期)
2. 让函数外部可以操作(读写)到函数内部的数据(变量/函数)

提问：

问题：

1. 函数执行完后，函数内部声明的局部变量是否还存在？
2. 在函数外部能直接访问函数内部的局部变量吗？

回答：

1,

一般是不会存在的，存在于闭包中的变量才可能会存在，而且需要将这个包含闭包的對象返回给一个变量存储

举例：

```
function fn(){
  var a=1;
  function fn2(){
    a++;
    console.log(a);
  }
  return fn2;
}
//var f=fn();      ---- 存在变量中闭包就存在
//fn();            ---- 直接使用的话就会立刻释放内存
```

2,

不能，但是可以通过闭包让外部操作它

举例：上个例子已经证明了闭包可以操作函数内部的局部变量

1.4.5 闭包的生命周期

1. 产生: 在嵌套内部函数定义执行完时就产生了(不是在调用)
2. 死亡: 在嵌套的内部函数成为垃圾对象时

-->

```
<script type="text/javascript">
  function fn1() {
    //此时闭包就已经产生了(函数提升, 内部函数对象已经创建了)
    var a = 2
    function fn2() {
      a++
      console.log(a)
    }
    return fn2
  }
  var f = fn1()
  f() // 3
  f() // 4
  f = null // 闭包死亡(包含闭包的函数对象成为垃圾对象)
```

情况二:

1. 产生: 在嵌套内部函数定义执行完时就产生了(不是在调用)
2. 死亡: 在嵌套的内部函数成为垃圾对象时

-->

```
<script type="text/javascript">
  function fn1() {
    var a = 2
    var fn2 = function () { //此时闭包就已经产生了(函数提升, 内部函数对象已经创建了)
      a++
      console.log(a)
    }
    return fn2
  }
  var f = fn1()
  f() // 3
  f() // 4
  f = null // 闭包死亡(包含闭包的函数对象成为垃圾对象)
```

1.4.6 闭包的应用

- * 具有特定功能的js文件
- * 将所有数据和功能都封装在一个函数内部(私有的)
- * 只向外暴露一个包含n个方法的对象或函数
- * 模块的使用者, 只需要通过模块暴露的对象调用方法来实现对应的功能

1.4.6.1 自定义 js 模块方法一：

```
<script>

function myModule(){
    var msg="frame";
    function doSth(){
        console.log("doSth");
    }
    function doOth(){
        console.log("doOth");
    }
    return{
        doSth:doSth,
        doOth:doOth
    }
}

var myModule2=new myModule();
myModule2.doSth();
myModule2.doOth();

</script>
```

1.4.6.2 自定义 js 模块二：使用匿名函数调用闭包

```
<script>

    (function(){
        var msg="frame1";

        function doSomething(){
            console.log("doSomething"+msg.toUpperCase());
        }

        function doOtherthing(){
            console.log("doOtherthing"+msg.toLowerCase());
        }

        window.myModule2={
            doSomething:doSomething,
            doOtherthing:doOtherthing
        }

    })()

    myModule2.doSomething();

    myModule2.doOtherthing();

</script>
```

两种自定义 js 模块的区别：第二种比第一种优秀，因为可以在脚本区域直接使用函数名变量进行使用，而第一种还必须要定义变量

1.4.7 闭包的缺点

闭包技术容易造成内存溢出和泄漏的问题

1. 缺点

- * 函数执行完后，函数内的局部变量没有释放，占用内存时间会变长
- * 容易造成内存泄露

2. 解决

- * 能不用闭包就不用
- * 及时释放

-->

```
<script type="text/javascript">
```

```
function fn1() {  
    var arr = new Array[100000]  
    function fn2() {  
        console.log(arr.length)  
    }  
    return fn2  
}  
var f = fn1()  
f()
```

```
f = null // 让内部函数成为垃圾对象-->回收闭包
```

1.4.7.1 内存溢出

1, 内存溢出

1. 内存溢出

- * 一种程序运行出现的错误
- * 当程序运行需要的内存超过了剩余的内存时，就抛出内存溢出的错误

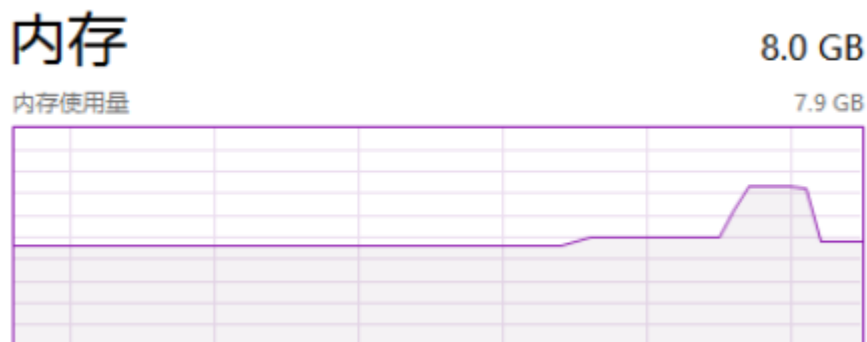
```
// 1. 内存溢出
```

```
var obj = {}  
for (var i = 0; i < 10000; i++) {  
    obj[i] = new Array(10000000)  
    console.log('-----')  
}
```

浏览器打印错误信息



电脑内存监控详情



1.4.7.2 内存泄漏

一块没有注意编码规范的代码容易出现部分内存泄漏，一个泄漏一点，最终导致内存溢出

2. 内存泄露

- * 占用的内存没有及时释放
- * 内存泄露积累多了就容易导致内存溢出
- * 常见的内存泄露：
 - * 意外的全局变量
 - * 没有及时清理的计时器或回调函数
 - * 闭包

三个举例代码：

```
//01 意外的全局变量
function fn(){
    a=1;//尽量避免这种不用 var 声明的变量
    console.log(a);
}
fn();

//02 启动循环定时器后不清理
var intervalId=setInterval(function(){
```

```

        console.log('-----');
    },1000);
//如果不做 clear 操作，就会产生内存泄漏
//clearInterval(intervalId);

```

//03 没有清理闭包导致的内存泄漏，因为局部变量 a 一直都存在于内存中

```

function fn1(){
    var a=4;
    function fn2(){
        console.log(a++);
    }
    return fn2;
}

var f=fn1();
f()
//不执行闭包清理就会导致内存泄漏
//f=fn2();
f=null;

```

1.4.8 闭包的一些难度较高的题目

题目一：

```

var name='windowname';

var object={
    name:'objectname',
    returnname:function(){
        return function(){
            return this.name; //this---window
        }
    }
}
//returnname是一个函数,一个返回的是一个执行函数的函数
//分开显示为
/*
    var fun=object.returnname();
    //执行函数
    fun()
    如果写在一起，就是object.returnname()()
*/
console.log(object.returnname()()); //windowname
//解释：
/*
    这里object.returnname(): 中的this代表的是object这个实例对象，但是object.returnname()执行的是一个函数
    而要得到这个函数的执行值，需要再加上（），但是本质上就等于先声明了一个函数名A，然后再console.log（）中
    打印console.log（A（）），这样的话，等同于window.A()，所以其实这里的this已经是window了
*/

```

题目二：使用闭包的原理，将 this 赋值给 that 变量，解决 this 指向的问题

```

var name='windowname';

var object={
  name:'objectname',
  returnname:function(){
    var that=this;//这里的this就是object
    return function(){
      return that.name; //that-this-object
    }
  }
}

console.log(object.returnname()); //objectname

```

题目三：闭包的终极面试题

```

//说在前面，有没有产生新的闭包，就看有没有执行函数本身 fun(n,o)
function fun(n,o){
  console.log(o);
  return { //这里其实是返回的一个对象
    fun:function(m){
      return fun(m,n);//这里会产生闭包
    }
  }
}

//n=0，然后内部函数产生了一个闭包，o 未定义，所以是 undefined，然后返回了一个对象
/*
    外部函数 fun(0,undefined)返回的是{fun(m)}的对象（这里的 fun 是内部函数 fun）

    因为并没有使用到这个内部函数 fun，所以现在不产生新的闭包
*/
var a=fun(0);//undefined
//因为返回的{fun(m)}对象被一个引用 a 保存，因此之前 fun（0）产生的闭包不会消失，所以 n 会一直保存 0 这个值
//接下来执行 a.fun(1)，实际上是{fun(m)}.fun(1)的执行，执行过程中这里会迅速产生一个新的闭包，返回过程中
//调用的是外部函数的 fun(n,o),传入参数是（1,0）--(这里的 n 还是之前闭包中的 0)--然后 console.log(o)--即打印 0
//然后因为没有把 a.fun(1)的返回值传给一个参数保存，因此第二次产生的闭包会迅速消失，接下来 n 中依然保存的是
//第一次执行的闭包结果，n 还是 0
a.fun(1);//0
a.fun(2);//0
a.fun(3);//0

var b=fun(0).fun(1).fun(2).fun(3);//u 0 1 2

var c=fun(0).fun(1);//0

```

```
c.fun(2);//1  
c.fun(3); //1
```

1.5 面向对象

1.5.1 对象创建模式

- h 01_Object构造函数模式.html
- h 02_对象字面量.html
- h 03_工厂模式.html
- h 04_自定义构造函数模式.html
- h 05_构造函数+原型的组合模式.html

1.5.2 继承模式

1.5.2.1 原型链继承

方式1: 原型链继承

1. 套路

1. 定义父类型构造函数
2. 给父类型的原型添加方法
3. 定义子类型的构造函数
4. 创建父类型的对象赋值给子类型的原型
5. 将子类型原型的构造属性设置为子类型
6. 给子类型原型添加方法
7. 创建子类型的对象: 可以调用父类型的方法

2. 关键

1. 子类型的原型为父类型的一个实例对象

默认方法一般都是 `MyObject.prototype=new Object();//⇔{ }`

//思考, 为什么一个自定义对象可以调用没有声明的toString方法

```
function MyObj(name){  
  this.name=name;  
}
```

```
var obj=new MyObj('apple');  
console.log(obj.toString());
```

//[object Object]----实际上用的是Object构造函数的显示原型对象中的toString方法

原型链继承就是使用的这个原理

```
function Fu(){
  this.name='Fu';
}
Fu.prototype.sayFu=function(){
  console.log(this.name+'hello');
}
function Son(){
  this.name='Son';
}
Son.prototype=new Fu();
//02看最后的注释
Son.prototype.constructor=Son;
Son.prototype.saySon=function(){
  console.log(this.name+'pei');
}
var son=new Son();
//01son.sayFu();//son.sayFu is not a function
son.sayFu();
//02
//这里会出现一个问题：就是new Son()其实是一个父类的实例
//需要在构造函数的地方将错误的值改过来
```

1.5.2.2 借用构造函数继承（不是真正意义上的继承）

讲解一下：这里的方法名.call 的意思是指定的函数体的执行，交给第一个参数传入的对象来执行，这里就是 this，就是后面 new 出来的 s

```
function Person(name, age) {
  this.name = name
  this.age = age
}
function Student(name, age, price) {
  Person.call(this, name, age) // 相当于: this.Person(name, age)
  /*this.name = name
  this.age = age*/
  this.price = price
}
var s = new Student('Tom', 20, 14000)
console.log(s.name, s.age, s.price)
```

1.5.2.3 组合继承

```
function Person(name, age) {  
  this.name = name  
  this.age = age  
}  
Person.prototype.setName = function (name) {  
  this.name = name  
}  
  
function Student(name, age, price) {  
  Person.call(this, name, age) // 为了得到属性  
  this.price = price  
}  
Student.prototype = new Person() // 为了能看到父类型的方法  
Student.prototype.constructor = Student // 修正constructor属性  
Student.prototype.setPrice = function (price) {  
  this.price = price  
}  
  
var s = new Student('Tom', 24, 15000)  
s.setName('Bob')  
s.setPrice(16000)  
console.log(s.name, s.age, s.price)
```

1.6 线程机制与事件机制

1.6.1 进程与线程

1, 进程

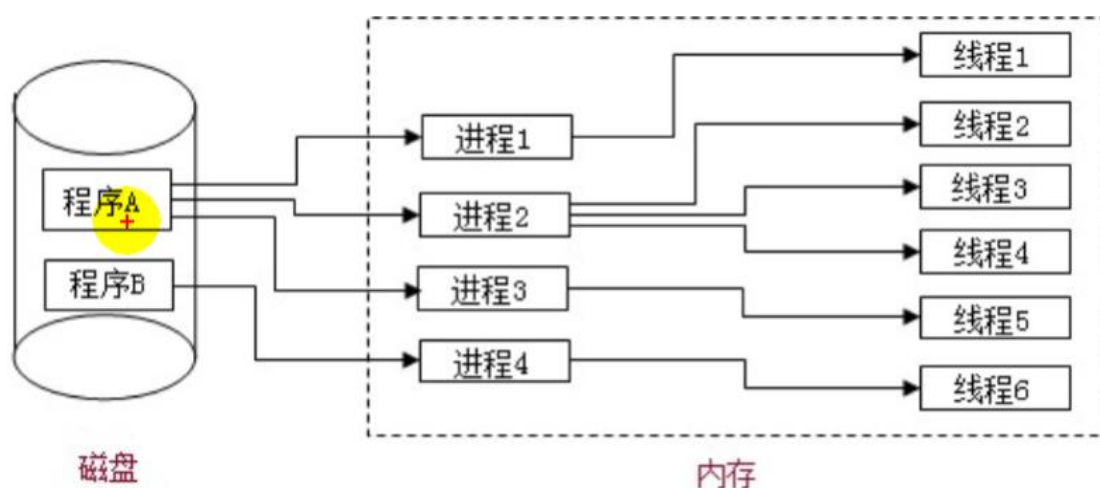
程序的一次执行, 它占有一片独有的内存空间
可以通过windows任务管理器查看进程

2, 线程

是进程内的一个独立执行单元

是程序执行的一个完整流程

是CPU的最小的调度单元



3, 相关知识

应用 程序必须运行在某个进程的某个线程上

一个进程中至少有一个运行的线程: 主线程, 进程启动后自动创建

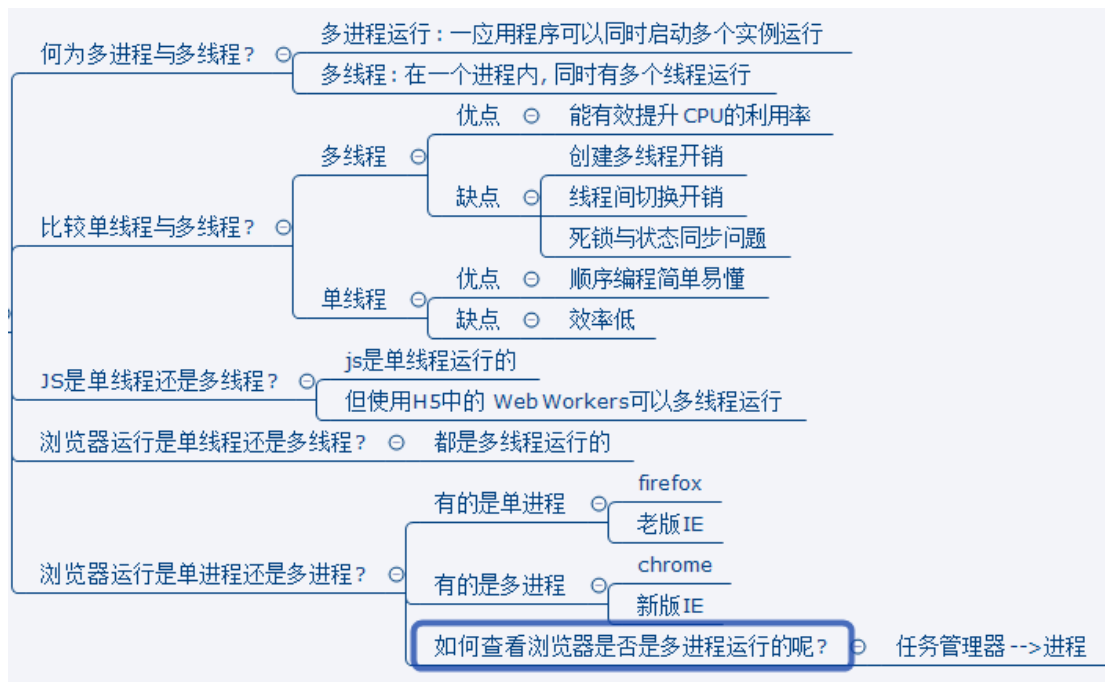
一个进程中也可以同时运行多个线程, 我们会说程序是多线程运行的

一个进程内的数据可以供其中的多个线程直接共享

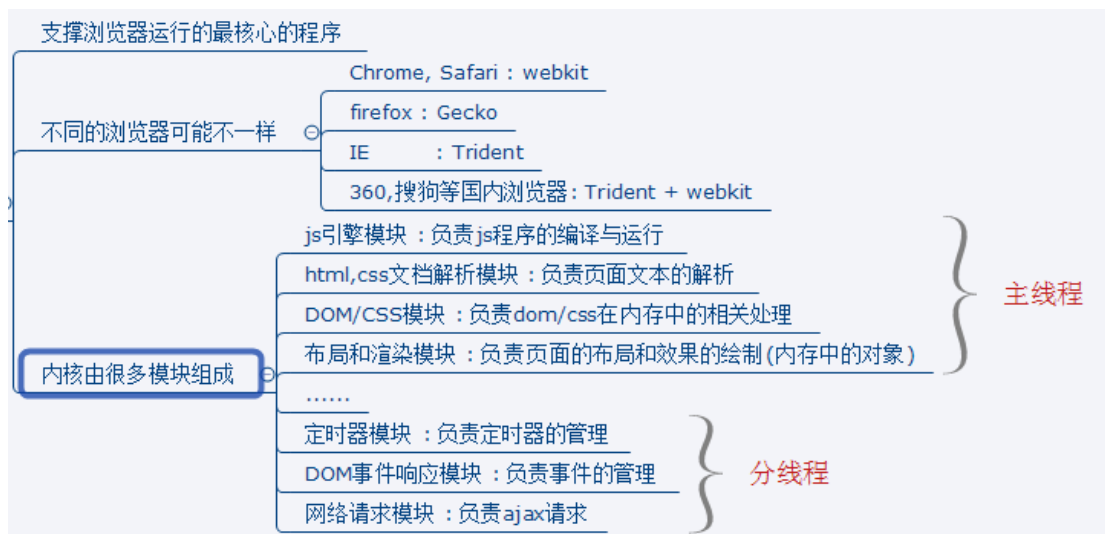
多个进程之间的数据是不能直接共享的

线程池(thread pool): 保存多个线程对象的容器, 实现线程对象的反复利用

4, 相关问题



1.6.2 浏览器内核



1.6.3 定时器引发的思考

1.6.4js 是单线程执行的

1.6.5 浏览器的事件循环（轮询）模型

1.6.6H5 Web Workers（多线程）

2 自定义框架

2.1js 自写框架

2.2 简易框架实现案例

1， 下载 jquery





2，实现案例

通过 js 实现

```
<title>无标题文档</title>
<script src="jquery-1.7.1.js"></script>
<script>
window.onload=function ()
{
    var oBtn=document.getElementById('btn1');

    oBtn.onclick=function ()
    {
        alert('a');
    };
};
</script>
</head>

<body>
<input id="btn1" type="button" value="按钮" />
</body>
```

通过 jquery 实现

```
$(function () {
    $('#btn1').click(function () {
        alert('a');
    });
});
```

\$是什么

```
alert(typeof $);
```

: function

Jquery 中的传入的参数是什么

```
$(function () {
    $('#btn1').click(function () {
        alert('a');
    });
});

$(function () {
    $('#btn1').click(function () {
        alert($(this).attr('value'));
    });
});
```

总结选择器\$:

- 1, 函数
- 2, 字符串 #xxx .xxx xxx
- 3, 对象

定义 vquery

```
<script src="VQuery.js"></script>
<script>
</head>
```

3 绑定事件，让函数可以执行多次

```
function myAddEvent(obj, sEv, fn)
{
    if(obj.attachEvent)
    {
        obj.attachEvent('on'+sEv, fn);
    }
    else
    {
        obj.addEventListener(sEv, fn, false);
    }
}

function VQuery(vArg)
{
    switch(typeof vArg)
    {
        case 'function':
            //window.onload=vArg;
            myAddEvent(window, 'load', vArg);
            break;
    }
}
```

Html

```
new VQuery(function () {
    alert('a');
});

new VQuery(function () {
    alert('b');
});
```

捕获字符串


```

default:      //tagName
    this.elements=document.getElementsByTagName (vArg);

```

实现点击事件

```

$('div').click(function () {
});

```

添加原型 click 方法

```

VQuery.prototype.click=function (fn)
{
    var i=0;

    for(i=0;i<this.elements.length;i++)
    {
        myAddEvent(this.elements[i], 'click', fn);
    }
};

```

执行事件响应案例：针对 id, class 和标签演示，这里只有 id

```

<script src="VQuery.js"></script>
<script>
new VQuery(function () {
    new VQuery('#btn1').click(function () {
        alert('a');
    });
});
</script>
</head>

<body>
<input id="btn1" type="button" value="aaa" />
</body>

```

添加对象选择

```

switch(typeof vArg)
{
    case 'function':
        //window.onload=vArg;
        myAddEvent(window, 'load', vArg);
        break;
    case 'string':
        switch(vArg.charAt(0))
        {
            case '#': //ID
                var obj=document.getElementById(vArg.substring(1));

                this.elements.push(obj);
                break;
            case '.': //class
                this.elements=getByClass(document, vArg.substring(1));
                break;
            default: //tagName
                this.elements=document.getElementsByTagName(vArg);
        }
        break;
    case 'object':
        this.elements.push(vArg);
}

```



改成美元符号

```

$(function () {
});

```

框架增加方法

```

function $(vArg)
{
    return new VQuery(vArg);
}

```

前端页面增加案例演示

```

$(function () {
    $('input').click(function () {
        alert('a');
    });
});

```

实现展开和隐藏

```

<head>
<style>
#div1 {width:100px; height:100px; background:red;}
</style>
<meta http-equiv="Content-Type" content="text/html; charset=utf-8" />
<title>无标题文档</title>
<script src="VQuery.js"></script>
<script>
$(function () {
    $('#btn1').click(function () {
        $('#div1').show();
    });

    $('#btn2').click(function () {
        $('#div1').hide();
    });
});
</script>
</head>

<body>
<input id="btn1" type="button" value="显示" />
<input id="btn2" type="button" value="隐藏" />
<div id="div1">
</div>
</body>

```

添加 show 和 hide 方法

```

VQuery.prototype.show=function ()
{
    var i=0;

    for(i=0;i<this.elements.length;i++)
    {
        this.elements[i].style.display='block';
    }
};

VQuery.prototype.hide=function ()
{
    var i=0;

    for(i=0;i<this.elements.length;i++)
    {
        this.elements[i].style.display='none';
    }
};

```

Hover: 移入移出

```

$('#div1').hover(function () {}, function () {});←

```

前十移入方法，后面是移出方法

```
VQuery.prototype.hover=function (fnOver, fnOut)
{
    var i=0;

    for(i=0;i<this.elements.length;i++)
    {
        myAddEvent(this.elements[i], 'mouseover', fnOver);
        myAddEvent(this.elements[i], 'mouseout', fnOut);
    }
};
```

案例演示

```
<script src="VQuery.js"></script>
<script>
$(function (){
    $('#div1').hover(function (){
        document.title='over了';
    }, function (){
        document.title='out了';
    });
});
</script>
```

Css 方法

```
$('#div1').css('background', 'red')    设置样式↓
$('#div1').css('width')                获取样式←
```

```
$('#div').css('backgroundColor')←
```

`$(selector).css(name)`

获得第一个匹配元素的样式属性值

```
VQuery.prototype.css=function (attr, value)
{
    if(arguments.length==2) //设置样式
    {
        var i=0;

        for(i=0;i<this.elements.length;i++)
        {
            this.elements[i].style[attr]=value;
        }
    }
    else //获取样式
    {
        return this.elements[0].style[attr];
    }
};
```

获取不了行间样式

```

<style>
#div1 {width:100px; height:100px; background:red;}
</style>
<meta http-equiv="Content-Type" content="text/html; charset=utf-8" />
<title>无标题文档</title>
<script src="VQuery.js"></script>
<script>
$(function (){
    /*$('#div1').hover(function (){
        //document.title='over';
        $('#div1').css('background', 'green');
    }, function (){
        //document.title='out';
        $('#div1').css('background', 'red');
    });*/
    $('#div1').click(function (){
        alert($('#div1').css('width'));
    });
</script>
</head>

<body>
<div id="div1">

```

行间样式可以获取解决:

```

<body>
<div id="div1" style="width:100px;">
</div>
</body>

```

加上 getstyle

```

function myAddEvent(obj, sEv, fn)
{ if(ob...

function getByClass(oParent, sClass)
{ var a...

function getStyle(obj, attr)
{
    if(obj.currentStyle)
    {
        return obj.currentStyle[attr];
    }
    else
    {
        return getComputedStyle(obj, false)[attr];
    }
}

```

修改 css 方法

```

VQuery.prototype.css=function (attr, value)
{
    if(arguments.length==2) //设置样式
    {
        var i=0;

        for(i=0;i<this.elements.length;i++)
        {
            this.elements[i].style[attr]=value;
        }
    }
    else //获取样式
    {
        //return this.elements[0].style[attr];
        return getStyle(this.elements[0], attr);
    }
};

```

演示获取样式

```

<style>
#div1 {width:100px; height:100px; background:red;}
</style>
<meta http-equiv="Content-Type" content="text/html; charset=utf-8" />
<title>无标题文档</title>
<script src="VQuery.js"></script>
<script>
$(function (){
    /*$('#div1').hover(function (){
        //document.title='over了';
        $('#div1').css('background', 'green');
    }, function (){
        //document.title='out了';
        $('#div1').css('background', 'red');
    });*/
    $('#div1').click(function (){
        alert($('#div1').css('width'));
    });
});
</script>
</head>

<body>
<div id="div1">
</div>
</body>

```

解决 this 问题在 ie 中出现的问题

一般不会这样写:

```

<script>
$(function (){
    $('#div1').hover(function (){
        $('#div1').css('background', 'green');
    }, function (){
        //document.title='out了';
        $('#div1').css('background', 'red');
    });
});
</script>

```

一般这样写，但是目前 this 在 ie 下会出问题

```
<script src="VQuery.js"></script>
<script>
$(function () {
    $('#div1').hover(function () {
        $(this).css('background', 'green');
    }, function () {
        $(this).css('background', 'red');
    });
});
</script>
```

问题出处:

```
function myAddEvent(obj, sEv, fn)
{
    if(obj.attachEvent) ←
    {
        obj.attachEvent('on'+sEv, fn);
    }
    else
    {
        obj.addEventListener(sEv, fn, false);
    }
}
```

这个会把被绑定的函数变成 window

讲解解决办法 call

```
<script>
function show(a, b)
{
    alert('this是: '+this+', a是: '+a+', b是: '+b);
}

show(12, 5); //window
show.call('abc', 12, 5); // 'abc'
show.apply('abc', [12, 5]);
</script>
```

修改事件绑定

```
function myAddEvent(obj, sEv, fn)
{
    if(obj.attachEvent)
    {
        obj.attachEvent('on'+sEv, function (){
            fn.call(obj);
        });
    }
    else
    {
        obj.addEventListener(sEv, fn, false);
    }
}
```

Toggle 的案例演示

```
<head>
<style>
#div1 {width:100px; height:100px; background:red; display:none;}
</style>
<meta http-equiv="Content-Type" content="text/html; charset=utf-8" />
<title>无标题文档</title>
<script src="jquery-1.7.1.js"></script>
<script>
$(function (){
    $('#input').toggle(function (){
        $('#div1').show();
    }, function (){
        $('#div1').hide();
    });
});
</script>
</head>

<body>
<input type="button" value="显示隐藏" />
<div id="div1">
</div>
</body>
```

原理：方法计算

0	fn1↓
1	fn2↓
2	fn1↓
3	fn2←

实现原理猜想

```

<script>
window.onload=function ()
{
    var aBtn=document.getElementsByTagName('input');
    var i=0;
    var count=0;

    for(i=0;i<aBtn.length;i++)
    {
        addClick(aBtn[i]);
    }

    function addClick(obj)
    {
        obj.onclick=function ()
        {
            alert(count++);
        };
    }
};
</script>

<body>
<input type="button" value="aaa" />
<input type="button" value="bbb" />
<input type="button" value="ccc" />
<input type="button" value="ddd" />
</body>

```

分别技术实现原理猜想问题解决

```

<script>
window.onload=function ()
{
    var aBtn=document.getElementsByTagName('input');
    var i=0;

    for(i=0;i<aBtn.length;i++)
    {
        addClick(aBtn[i]);
    }

    function addClick(obj)
    {
        var count=0;
        obj.onclick=function ()
        {
            alert(count++);
        };
    }
};
</script>
</head>

<body>
<input type="button" value="aaa" />
<input type="button" value="bbb" />
<input type="button" value="ccc" />
<input type="button" value="ddd" />
</body>

```

函数表现的两种方法

```
function aaaa()↓
{↓
    alert('a')↓
}↓
↓
oDiv.onclick=aaaa;↓
↓
-----↓
↓
oDiv.onclick=function ()↓
{↓
    alert('a')↓
};← I
```

改成匿名函数

```
<script>
window.onload=function ()
{
    var aBtn=document.getElementsByTagName('input');
    var i=0;

    for(i=0;i<aBtn.length;i++)
    {
        (function (obj){
            var count=0;
            obj.onclick=function ()
            {
                alert(count++);
            };
        })(aBtn[i]);
    }
};
</script>
```

最终的匿名函数版本

```

<script>
window.onload=function ()
{
    var aBtn=document.getElementsByTagName('input');
    var i=0;

    for(i=0;i<aBtn.length;i++)
    {
        aBtn[i].onclick=(function (count){
            return function (){
                alert(count++);
            };
        })(0);
    }
};
</script>

```

让自定义框架实现 toggle 点击计数

```

VQuery.prototype.toggle=function ()
{
    var i=0;

    for(i=0;i<this.elements.length;i++)
    {
        addToggle(this.elements[i]);
    }

    function addToggle(obj)
    {
        var count=0;
        myAddEvent(obj, 'click', function (){
            alert(count++);
        });
    }
};

```

案例演示

```

<script src="VQuery.js"></script>
<script>
$(function (){
    $('input').toggle();
});
</script>
</head>

```

I

```

<body>
<input type="button" value="1" />
<input type="button" value="2" />
<input type="button" value="3" />
<input type="button" value="4" />
</body>

```

升级方法使用规则，传递函数实现

```

<script>
$(function () {
    $('input').toggle(function () {
        alert('a');
    }, function () {
        alert('b');
    }, function () {
        alert('c');
    });
});
</script>

```

修改方法但是会出现问题，原因就是和 this 的闭包产生的问题相同，解决方法类比 that=this

```

VQuery.prototype.toggle=function ()
{
    var i=0;

    for(i=0;i<this.elements.length;i++)
    {
        addToggle(this.elements[i]);
    }

    function addToggle(obj)
    {
        var count=0;
        myAddEvent(obj, 'click', function () {
            arguments[count++].call(obj);
        });
    }
};

```

解决方法

```

VQuery.prototype.toggle=function ()
{
    var i=0;
    var _arguments=arguments;

    for(i=0;i<this.elements.length;i++)
    {
        addToggle(this.elements[i]);
    }

    function addToggle(obj)
    {
        var count=0;
        myAddEvent(obj, 'click', function () {
            _arguments[count++].call(obj);
        });
    }
};

```

案例演示

```
<style>
#div1 {width:100px; height:100px; background:red;}
</style>
<meta http-equiv="Content-Type" content="text/html; charset=utf-8" />
<title>无标题文档</title>
<script src="VQuery.js"></script>
<script>
$(function (){
    $('input').toggle(function (){
        $('#div1').hide();
    }, function (){
        $('#div1').show();
    });
});
</script>
</head>

<body>
<input type="button" value="显示隐藏" />
<div id="div1">
</div>
</body>
```

框架实现样式获取

```
~
VQuery.prototype.attr=function (attr, value)
{
    if(arguments.length==2)
    {
        var i=0;

        for(i=0;i<this.elements.length;i++)
        {
            this.elements[i][attr]=value;
        }
    }
    else
    {
        return this.elements[0][attr];
    }
}
```

效果案例演示

```

<script src="VQuery.js"></script>
<script>
$(function () {
    $('#btn1').click(function () {
        alert($('#txt1').attr('value'));
    });

    $('#btn2').click(function () {
        $('#txt1').attr('value', 'sdfsadf')
    });
});
</script>
</head>

<body>
<input id="txt1" type="text" />
<input id="btn1" type="button" value="读取文字" />
<input id="btn2" type="button" value="设置文字" />
</body>

```

Eq 的使用案例

```

<style>
div {width:100px; height:100px; background:red; float:left; margin:10px;}
</style>
<meta http-equiv="Content-Type" content="text/html; charset=utf-8" />
<title>无标题文档</title>
<script src="jquery-1.7.1.js"></script>
<script>
$(function () {
    $('#div').eq(2).css('background', 'green');
});
</script>
</head>

<body>
<div></div>
<div></div>
<div></div>
<div></div>
<div></div>
<div></div>
<div></div>
<div></div>
</body>
</html>

```

框架添加 eq 方法

```

VQuery.prototype.eq=function (n)
{
    return $(this.elements[n]);
};

```

实现 find 方法

原理:

```
<ul id="ul1">↓
  <li></li>↓
  <li></li>↓
  <li></li>↓
</ul>↓
<ul>↓
  <li class="box"></li>↓
  <li class="box"></li>↓
  <li></li>↓
</ul>↓
<ol>↓
  <li></li>↓
  <li></li>↓
  <li></li>↓
</ol>↓
↓
$('ul').find('li'); //6↑↓
$('ol').find('li'); //3↓
$('#ul1').find('li'); //3↓
↓
$('ul').find('.box'); //2↓
↓
$('#div1').find('#ul1'); ↓
```

```
function appendArr(arr1, arr2)
{
  var i=0;

  for(i=0;i<arr2.length;i++)
  {
    arr1.push(arr2[i]);
  }
}
```

```

VQuery.prototype.find=function (str)
{
    var i=0;
    var aResult=[];

    for(i=0;i<this.elements.length;i++)
    {
        switch(str.charAt(0))
        {
            case '.': //class
                var aEle=getByClass(this.elements[i], str.substring(1));

                aResult=aResult.concat(aEle);
                break;
            default: //标签
                var aEle=this.elements[i].getElementsByTagName(str);

                //aResult=aResult.concat(aEle);
                appendArr(aResult, aEle);
        }
    }

    var newVquery=$();

    newVquery.elements=aResult;

    return newVquery;
};

```

案例实现

```

<script>
$(function (){
    $('#ul1').find('li').css('background', 'red');
});
</script>
</head>

<body>
<ul id="ul1">
    <li></li>
    <li></li>
    <li></li>
</ul>
<ul>
    <li class="box"></li>
    <li class="box"></li>
    <li></li>
</ul>

```

2.3 常规框架实现案例

